

MMORPG-with-IOCP

게임서버포트폴리오—IOCP기반대규모동시접속서버 설계

개발자	김성준 (서버 단독 설계 · 구현)
개발 기간	약 3개월 (퇴사 후, 4월 ~ 6월경)
개발 형태	개인 프로젝트 · 경력 1년차 시점에 전면 리팩토링
Repository	github.com/SJKIM99/MMORPG-with-IOCP

IOCP · Zone 기반 액터 모델 · Lock-free 패킷 디스패치 · 자체 부하테스트 툴

목차

- 01 프로젝트 개요
 - 02 전체 시스템 아키텍처
 - 03 네트워크 계층 — IOCP Worker Thread
 - 04 핵심 설계 — Zone 기반 액터 모델
 - 05 Sector 기반 시야 최적화
 - 06 Lock-free 패킷 디스패치 풀
 - 07 성능 검증 — 자체 부하테스트
 - 08 담당 범위와 기술적 판단
 - 09 마무리
-

배경

이 프로젝트의 서버는 대학교 3학년 학기 중 텀프로젝트로 처음 만들어졌다. 이후 게임 서버 개발자로 약 1년간 실무 경험을 쌓은 시점에, 텀프로젝트 당시의 구조를 그대로 두지 않고 처음부터 끝까지 본인의 방식으로 전면 리팩토링했다. 클라이언트(SFML 기반)는 학교에서 교수님께서 제공해주신 기본 뼈대를 그대로 사용했으며, 본인은 그래픽적인 부분만 일부 수정했다. 본인이 서버 개발자로서 커리어를 쌓아온 만큼 클라이언트의 비주얼보다는 서버 아키텍처 자체의 완성도를 우선순위로 두고 진행한 프로젝트다.

목표

- 실무에서 다뤄본 IOCP 기반 동시성 처리 구조를 개인 프로젝트로 재현하고 확장
- 동시접속자 수가 늘어날 때 실제로 발생하는 병목을 직접 진단하고 구조적으로 해결
- 자체 제작한 부하테스트 툴로 개선 효과를 수치로 검증

개발 정보

개발 기간	약 3개월 (퇴사 후, 4월 ~ 6월경)
개발 인원	1인 (서버 설계·구현 전체 담당)
프로젝트 출발점	대학교 3학년 텀프로젝트 → 경력 1년차 시점에 서버 전면 리팩토링
담당 범위	서버 전 모듈(Network / Thread / Zone / Sector / Monster / Route / DB) 설계·구현
클라이언트	SFML 기반, 학교 제공 뼈대 + 그래픽 일부 수정

기술 스택

언어 / 플랫폼	C++, Windows IOCP
동시성	Worker Thread Pool, Zone 단위 로직 스레드, Lock-free SLIST
데이터베이스	MSSQL, ODBC (파라미터 바인딩 쿼리, 커넥션 풀)
클라이언트	SFML 2.5.1
검증 도구	자체 제작 STRESS_TEST 봇 툴

02 전체 시스템 아키텍처

클라이언트의 요청은 IOCP 기반 Network I/O 계층에서 비동기로 수신되고, 파싱된 패킷은 Dispatch Pool(Lock-free SLIST 기반)에서 재사용 가능한 작업 객체로 감싸져 담당 Zone의 큐로 전달된다. 각 Zone은 자신이 소유한 Sector·오브젝트에 대해 단독으로 로직을 처리하는 액터(Actor) 형태로 동작한다. 이와 별도로 Timer Thread가 예약된 이벤트(스킬 쿨다운, 버프 만료, 리스폰 타이머 등)의 만료 시점을 감시하다가, 이벤트가 만료되면 해당 오브젝트를 소유한 Zone Thread로 처리 요청을 넘긴다. DB 요청은 별도의 DB 스레드 풀을 통해 비동기적으로 처리되어 로직 스레드가 DB I/O로 블로킹되지 않도록 설계했다.

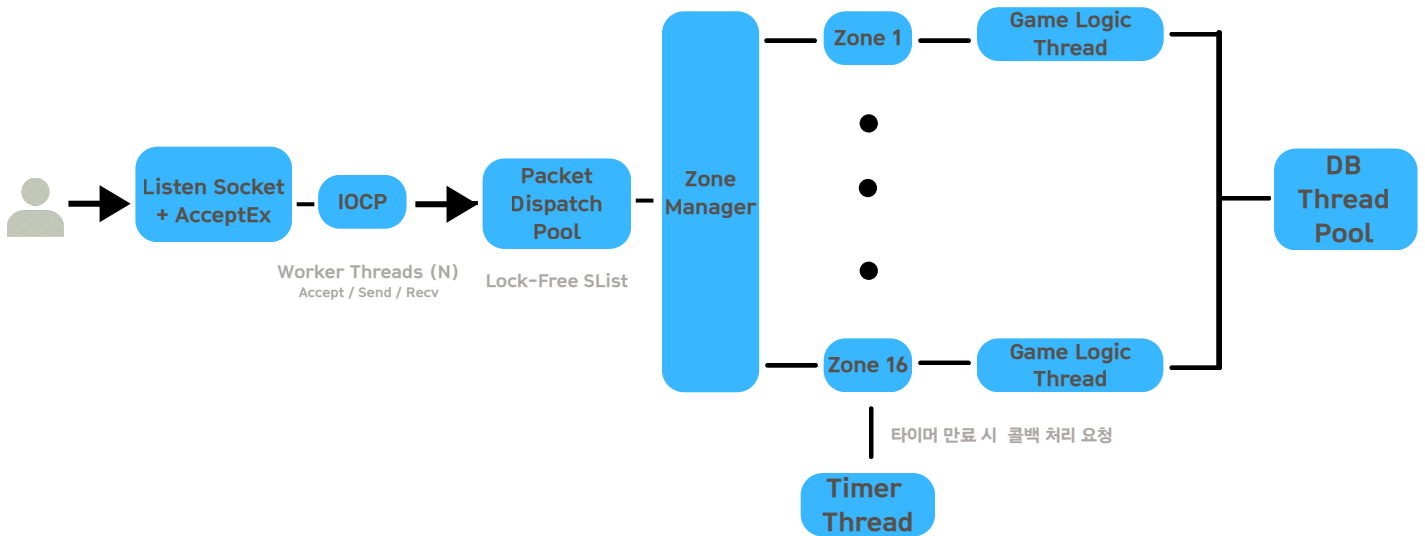
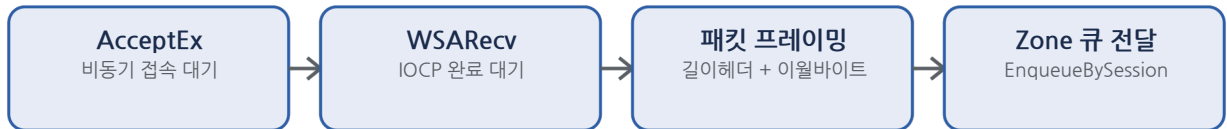


그림 2-1. 전체 시스템 구성도 — Network I/O · Zone 스레드 · DB 스레드 풀

계층별 역할

Network I/O	AcceptEx / WSARcv / WSASend를 IOCP 워커 스레드에서 처리, 패킷 프레임링
Dispatch Pool	수신된 패킷을 재사용 가능한 작업 객체로 감싸 담당 Zone 큐로 전달하는 lock-free 풀
Zone Thread	공간을 분할한 Zone 단위로 스레드를 할당, 해당 Zone의 오브젝트 로직을 단독 처리
Sector	Zone 내부의 세부 공간 분할 단위 — 시야 판정을 통해 불필요한 틱 소비를 줄임
Timer Thread	스킬 쿨다운·버프·리스폰 등 예약 이벤트의 만료 시 해당 Zone Thread로 처리를 위임
DB Thread Pool	커넥션 풀 기반으로 쿼리를 비동기 처리, 로직 스레드의 블로킹 방지

Windows IOCP(I/O Completion Port)를 사용해 하나의 완료 포트에 다수의 워커 스레드를 붙이는 구조로 비동기 네트워크 I/O를 처리했다. AcceptEx로 비동기 접속을 계속 게시해두고, GetQueuedCompletionStatus 루프에서 recv/send 완료 통지를 받아 처리한다. TCP_NODELAY를 설정해 Nagle 알고리즘으로 인한 지연을 없앴고, 패킷은 길이 헤더 기반으로 프레이밍하되 한 번의 recv로 패킷 경계가 딱 맞아떨어지지 않는 경우(부분 수신)를 대비해 남은 바이트를 다음 처리로 이월하는 방식으로 구현했다.



GetQueuedCompletionStatus() 루프 — 4단계가 워커 스레드 안에서 반복됨

그림 3-1. IOCP Worker Thread의 패킷 처리 흐름

설계 포인트

- 0바이트 recv/send(연결 종료 신호)를 별도로 분기 처리해 세션을 정상적으로 정리
- 세션 상태(전송 중/종료 중)에 락을 걸어 멀티 워커 스레드에서 동시에 같은 세션에 접근해도 안전하게 처리
- 패킷 디스패치 객체는 매번 new/delete 하지 않고 lock-free 풀에서 재사용 (6번 항목에서 상세 설명)

문제 인식

초기 구조는 Zone 개념 없이 Sector만 존재했고, 게임 로직 전체를 단일 GameLogicThread 하나가 순차적으로 처리했다. 스트레스테스트로 봇을 계속 붙여보면서, 동시접속자가 늘어날수록 몬스터의 AStar 경로 생성이 빈번해지고 봇들도 주기적으로 로직을 수행하다 보니 이 단일 로직 스레드에 병목이 심해지는 것을 체감했다.

해결 방향

이 병목을 풀기 위해 Zone이라는 개념을 새로 도입했다. Zone마다 전용 스레드를 하나씩 할당하고, 각 Zone이 자신의 오브젝트에 대한 로직을 자체적으로 처리하도록 바꿔서 하나의 스레드가 모든 부하를 떠안는 구조를 없앴다.

Before : 단일 로직 스레드 + Sector

After : Zone별 스레드 분산



체감한 문제

동접 증가 시 몬스터 AStar 경로 생성 빈도 증가 + 스트레스테스트 봇의 주기적 로직 처리로
단일 게임로직스레드에 병목이 체감됨 → Zone 단위 스레드 분리로 해결

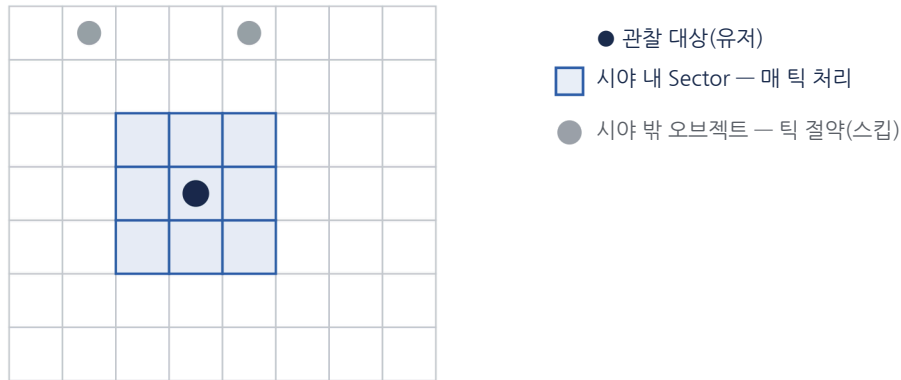
그림 4-1. Zone 도입 전후 구조 비교

결과

Zone 도입 후 STRESS_TEST 봇 기준 동시접속 처리량이 4,000명에서 16,000명 수준으로 개선됐다. 정확한 수치는 7장(성능 검증)에서 다룬다.

05 Sector 기반 시야 최적화

Zone과 Sector는 역할이 다르다. Zone은 스프레드를 나누는 단위이고, Sector는 그 안에서 공간을 더 잘게 쪼갠 시야 판정 단위다. 유저 시야에 들어오지 않는 오브젝트까지 매 틱 깨워서 처리할 필요는 없기 때문에, Sector 단위로 시야 판정을 해서 굳이 깨울 필요가 없는 오브젝트는 틱 처리에서 제외하는 방식으로 불필요한 연산을 아꼈다.

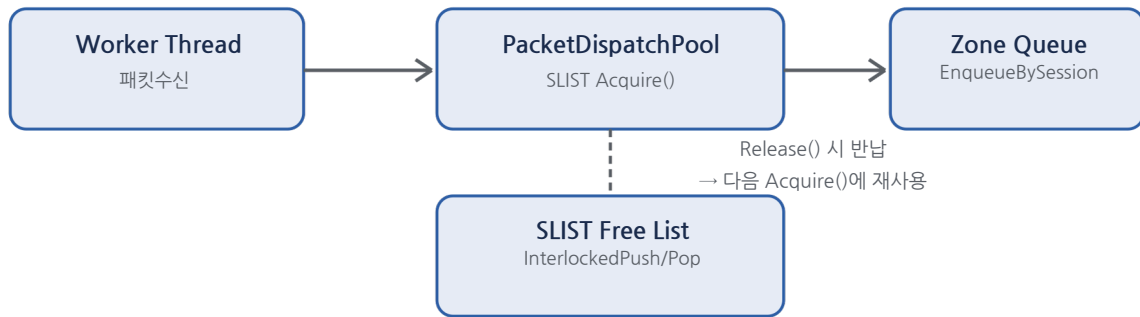


Zone = 스프레드 소유 단위 · Sector = 공간 분할·시야 판정 단위

그림 5-1. Sector 기반 시야 판정 개념도

이 구조 덕분에 Zone 스프레드 하나가 담당하는 오브젝트 수가 늘어나도, 실제로 매 틱 연산이 필요한 대상은 시야 안에 있는 오브젝트로 제한되어 Zone 도입 효과를 극대화할 수 있었다.

워커 스레드에서 받은 패킷을 비즈니스 로직 처리를 위해 Zone 큐로 넘기는 과정에서, 매번 객체를 new/delete 하는 대신 재사용 가능한 디스패치 객체 풀을 만들기로 했다. 처음에는 락의 오버헤드 자체를 없애고 싶어서 완전한 lock-free 큐로 구현을 시도했지만, 이 방식에서는 ABA 문제가 발생할 수 있다는 걸 예상했고, 이를 피하기 위해 Windows의 SLIST(Interlocked Singly Linked List) 기반 freelist 구조로 다시 설계했다.



락 부하 제거 목적 → 초기 lock-free 큐 시도 → ABA 문제 예상 → SLIST freelist로 전환
그림 6-1. 패킷 디스패치 풀의 Acquire / Release 흐름

이 과정에서 얻은 것은 단순히 '락을 안 쓴다'는 결과가 아니라, 락 프리 구조를 시도하면서 ABA 문제라는 구체적인 위험을 먼저 예상하고, 그 위험을 회피할 수 있는 대안(freelist)으로 스스로 판단해 전환했다는 과정 자체다.

07 성능 검증 — 자체 부하테스트

개선 효과를 감으로 판단하지 않기 위해 STRESS_TEST라는 부하테스트 툴을 직접 만들어 검증했다. 봇 클라이언트를 계속 붙여가며 서버 CPU 사용률을 관찰하고, CPU 사용률이 튀면 봇을 늘리는 속도를 줄이고, 다시 안정되면 늘리는 속도를 높이는 방식으로 서버가 감당할 수 있는 최대 동접 수준을 확인했다.

STRESS_TEST 봇 기준 동접 처리량

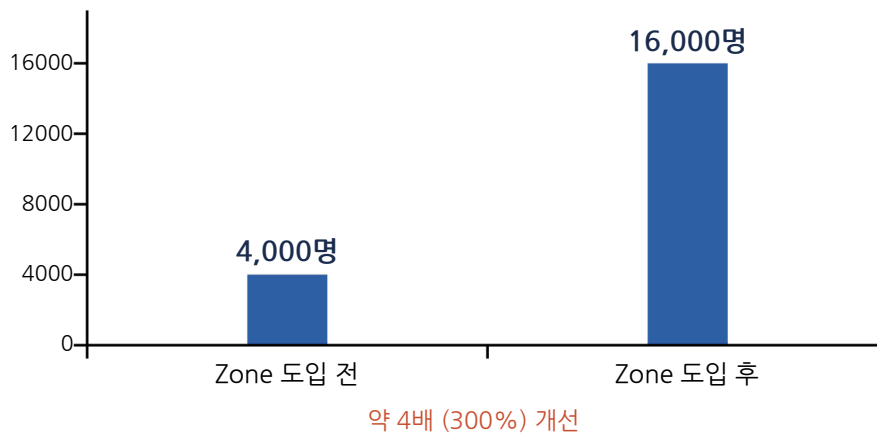


그림 7-1. Zone 도입 전후 동시접속 처리량 (STRESS_TEST 봇 기준)

테스트 스펙

봇 행동 패턴

행동	주기	내용
이동	400~600ms 랜덤	8방향(상·하·좌·우·대각) 중 이동 가능한 방향 랜덤 선택
일반 공격	400~600ms 랜덤	시야 내 몬스터 존재 시 공격 패킷 전송, facing 방향 반영
스킬 공격	5,200~7,000ms 랜덤	시야 내 몬스터 존재 시 스킬 패킷 전송
채팅	10,000~20,000ms 랜덤	7종 메시지 중 랜덤 전송 ("hello", "what's your name" 등)

부하 제어 알고리즘 (3단계 자동 조절)

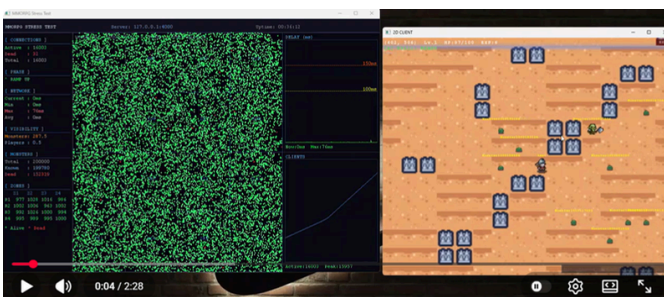
RAMP_UP → 접속 속도 50ms 간격으로 봇 추가

STABLE → 지연 100ms 초과 시, 접속 속도 500ms로 감속

REDUCING → 지연 150ms 초과 시, 봇 1개씩 강제 종료

- 서버가 회복돼 안정 상태(STABLE 100회 연속)가 되면 → 다시 RAMP_UP 재개
- 최고 부하 도달 시 maxLimit을 기록, 그 95% 이내로 수렴

시연 영상 ← 클릭시 이동



담당 범위

Network, Thread, Core, Zone/Sector, Monster, Route, DB 등 서버의 전 모듈을 혼자 설계하고 구현했다. 클라이언트는 학교에서 교수님께서 제공해주신 SFML 기반 뼈대를 그대로 사용했고, 본인은 그래픽 표현 일부만 수정했다.

클라이언트를 직접 새로 만들지 않은 이유

이 프로젝트를 진행하던 컴퓨터 환경으로는 게임 엔진을 사용한 클라이언트 구성이 어려웠고, 본인이 서버 개발자로 커리어를 쌓아온 만큼 클라이언트의 비주얼적인 부분은 우선순위가 아니라고 판단했다. 그래서 검증된 학교 산출물 뼈대를 그대로 활용하고, 리소스를 서버 아키텍처 설계와 성능 개선에 집중적으로 투입했다.

텀프로젝트에서 출발한 부분에 대해

서버의 최초 출발점은 대학교 3학년 텀프로젝트가 맞다. 다만 이후 경력 1년차 시점에 처음부터 끝까지 본인의 방식으로 전면 리팩토링을 진행했으며, 이번 포트폴리오에서 다루는 Zone 기반 액터 모델, Sector 시야 최적화, Lock-free 디스패치 풀, 자체 부하테스트 툴 등은 모두 이 리팩토링 과정에서 새로 설계하고 추가한 부분이다.

이 프로젝트를 통해 이론으로만 알던 동시성 문제(락 경쟁, ABA 문제, 메모리 오더링)를 실제로 구조에 부딪히며 해결해봤다. 특히 단일 스레드 구조에서 병목을 체감하고, 그 원인을 구조적으로 분석해 Zone 단위 분산이라는 해결책을 스스로 설계하고, 이를 자체 제작한 부하테스트 툴로 수치까지 검증해본 경험이 가장 큰 성과라고 생각한다.

이번 프로젝트에서 확인한 강점

- IOCP 기반 비동기 네트워크 처리와 멀티스레드 동시성 설계에 대한 실무 수준의 이해
- 체감한 병목을 구조적 원인으로 좁혀나가고, 대안을 설계해 직접 구현하는 문제 해결 방식
- 감이 아니라 자체 제작 도구로 수치를 남겨 개선 효과를 검증하는 습관

Repository : github.com/SJKIM99/MMORPG-with-IOCP